

Modul 4 — Isolasi Multi-Tenant: Context, Scope, Policy, dan Binding

Pertemuan	Minggu 4 dari 14
Fase	Fase 1 — Keamanan data
Durasi	210 menit efektif di kelas (3 jam 30 menit); tugas mandiri di luar kelas hingga 120 menit (2 jam)
Ketertelusuran	FR-TEN-01; matriks konteks akses Lampiran B; NFR keamanan dan privasi

Output utama: TenantContext request-scoped dan middleware resolver.

Tujuan Pembelajaran

- ☐ Membangun TenantContext request-scoped. (C3 – Menerapkan)
- ☐ Menerapkan trait global scope dan auto-fill tenant_id. (C3 – Menerapkan)
- ☐ Menggabungkan middleware, policy, route binding, dan query publik. (C4 – Menganalisis)
- ☐ Menulis security regression test untuk akses lintas tenant. (C5 – Mengevaluasi)

Output Project

- ☐ TenantContext request-scoped dan middleware resolver.
- ☐ Trait `BelongsToTenant`` pada model tenant-owned.
- ☐ Policy dan scoped route binding untuk menu/order/withdrawal.
- ☐ Test matrix allow/deny tenant A vs tenant B.

Pendahuluan

Pada Modul 3 Anda sudah memasang foreign key komposit yang mencegah relasi lintas tenant di tingkat database. Lapisan itu perlu, tetapi belum cukup: query biasa masih dapat membaca data tenant lain bila aplikasi lupa menyaring. Multi-tenancy shared database — satu database dipakai bersama banyak tenant — aman hanya bila semua jalur akses mematuhi tenant aktif. Minggu ini menerapkan pertahanan berlapis: TenantContext dari route atau role pengguna, global scope pada model tenant-owned, policy untuk aksi, scoped route binding, composite foreign key, dan test negatif lintas tenant.

Global scope — filter yang otomatis ditambahkan Eloquent pada setiap query sebuah model — bukan satu-satunya keamanan. Query publik katalog memang perlu melewati scope untuk menampilkan banyak tenant, tetapi harus mengganti pengaman itu dengan filter canteen dan status. Semua penggunaan `withoutGlobalScopes()` harus eksplisit serta direview.

Pengetahuan Teknis dan Penjelasan Mandiri (Laravel 13)

FOKUS TEKNIKIS • TenantContext per request/job dan pertahanan berlapis pada query, route, serta aksi pengguna.

- ☐ **Siklus hidup context.** Bind `TenantContext`` dengan `scoped()` — satu instance yang hidup selama satu request atau satu job saja — bukan singleton proses yang bertahan selama server berjalan. Scoped binding di-reset pada request baru dan lifecycle job queue baru, sehingga worker panjang tidak membawa tenant sebelumnya [L13-DI].
- ☐ **Resolver middleware.** Autentikasi harus selesai sebelum middleware memetakan user, host, atau parameter route ke tenant aktif. Tolak request jika identitas tenant hilang, nonaktif, atau tidak dimiliki aktor [L13-MW] [L13-AUTHN].

- ❑ **Global scope.** Scope Eloquent menambahkan filter default untuk model tenant-owned. ``withoutGlobalScope()`` hanya boleh dipakai pada alur lintas tenant yang diotorisasi dan wajib diganti filter `kantin/tenant/status` eksplisit [L13-ORM].
- ❑ **Binding dan policy.** Aktifkan scoped implicit binding dengan ``scopeBindings()`` untuk resource bersarang, lalu gunakan policy/middleware ``can`` untuk memutuskan aksi. Binding membatasi objek; policy membatasi kemampuan [L13-ROUTE] [L13-AUTHZ].
- ❑ **Queue dan pengujian negatif.** Job harus membawa ``tenant_id`` eksplisit dan membentuk context sebelum query. Uji Tenant A terhadap ID Tenant B untuk index, show, update, delete, route binding, serta job [L13-QUEUE] [L13-HTTPTEST].

TUJUAN PEMAHAMAN · Isolasi tenant harus berupa pertahanan berlapis. Tidak ada satu middleware atau satu global scope yang cukup untuk seluruh jalur web, job, console, dan query publik. Rujukan tetap dicantumkan untuk traceability, bukan sebagai prasyarat memahami materi.

Membentuk TenantContext dengan urutan yang benar. Request terlebih dahulu dikenali oleh router dan middleware authentication. Resolver kemudian membaca host, parameter route, atau pilihan tenant milik user, memverifikasi status serta membership, lalu mengisi ``TenantContext``. Context memakai binding ``scoped`` agar satu request memperoleh nilai konsisten, tetapi nilai dibuang sebelum request/job berikutnya. Singleton berbahaya pada queue worker atau server panjang karena objek dapat membawa tenant sebelumnya. Jika context belum terisi, query tenant-owned harus gagal tertutup (fail closed — menolak akses saat ragu, bukan mengizinkan), bukan diam-diam menjadi query lintas tenant [L13-DI] [L13-MW].

Empat lapis pembatasan. Global scope menambahkan ``where tenant_id = ...`` pada query normal. Scoped route binding memastikan resource child ditemukan di bawah parent route yang benar. Policy memeriksa kemampuan aktor terhadap objek dan keadaan bisnis. Foreign key/unique constraint menjaga integritas ketika kode aplikasi salah atau dua transaksi berlomba. Keempat lapisan menyelesaikan risiko yang berbeda: query, resolusi URL, keputusan aksi, dan konsistensi penyimpanan. Hasil yang diharapkan pada percobaan lintas tenant adalah 404 atau 403 tanpa membocorkan data [L13-ORM] [L13-ROUTE] [L13-AUTHZ] [L13-MIG].

Bypass scope yang terkontrol. ``withoutGlobalScope()`` bukan izin akses; ia hanya mengubah bentuk query. Bypass dipakai oleh alur yang memang lintas tenant, misalnya katalog publik satu kantin atau laporan admin platform. Sebelum bypass, service harus memperoleh canteen/tenant yang telah diotorisasi; setelah bypass, query wajib menambahkan filter pengganti yang eksplisit. Letakkan query semacam ini pada class khusus, beri nama yang menunjukkan lingkupnya, dan uji bahwa tenant tidak aktif atau tenant kantin lain tidak ikut terbaca. Bypass tersebar di controller sulit diaudit [L13-ORM] [L13-QB].

Context pada queue dan test negatif. Job tidak membawa object context dari request. Payload job menyimpan ``tenant_id`` dan ID resource minimum; pada ``handle()``, job memuat tenant, memverifikasi masih aktif, lalu membentuk context sebelum query domain. Hindari serialisasi model penuh karena data dan scope dapat berubah sebelum job diproses. Test negatif harus mencoba ID milik Tenant B melalui index, detail, update, delete, binding, export, broadcast channel, dan job yang dijalankan sebagai Tenant A. Assertion tidak cukup hanya memeriksa status; pastikan data B tidak berubah dan tidak tampil pada response [L13-QUEUE] [L13-HTTPTEST].

LANDASAN TEKNIS — AppServiceProvider.php + TenantScope.php • PHP

```
// app/Providers/AppServiceProvider.php
public function register(): void
{
    $this->app->scoped(
        TenantContext::class,
        fn (): TenantContext => new TenantContext(),
    );
}

// app/Models/Concerns/BelongsToTenant.php
public function apply(Builder $builder, Model $model): void
{
    $tenantId = app(TenantContext::class)->id();

    $builder->where(
        $model->qualifyColumn('tenant_id'),
        $tenantId,
    );
}
```

Jembatan dari Mata Kuliah Prasyarat

HUBUNGAN · Dari query berdasarkan ID menuju isolasi berlapis: TenantContext, global scope, policy, dan scoped route binding.

Sebelum Coding: Pause & Predict

- Mengapa global scope saja belum cukup?
- Kapan bypass scope sah?
- Bagaimana mencegah context bocor pada queue worker?

Tulis jawaban sementara maksimal tiga kalimat per pertanyaan. Revisi jawaban setelah fase Observe dan tandai perubahan pemahaman.

Prasyarat

- ☐ Model, migrasi, dan seeder dari Modul 3 sudah berjalan; minimal tersedia dua tenant agar pengujian lintas tenant dapat dilakukan.
- ☐ Daftar platform-scoped vs tenant-owned telah disetujui.

Rencana Waktu — 3 Jam 30 Menit Efektif di Kelas

Tahap	Menit	Aktivitas	Checkpoint
1	30	Definisikan konteks dan aturan akses	Setiap bypass mempunyai filter pengganti dan alasan.
2	30	Implementasikan TenantContext	Dua request/job tidak berbagi state tenant.
3	30	Middleware resolver dan role check	Tenant yang suspended atau tanpa role mendapat 403.
4	30	Global scope dan model integration	Query biasa hanya mengembalikan tenant aktif.
5	30	Policy dan scoped route binding	Mengubah ID pada URL atau payload Livewire tidak membuka data lain.
6	30	Public query dan bypass terkontrol	Katalog hanya menampilkan tenant aktif dari canteen hasil QR.
7	30	Security tests dan commit	Seluruh matriks allow/deny hijau.

Total efektif di kelas: 210 menit (7 tahap × 30 menit). Tugas mandiri di luar kelas: hingga 120 menit (2 jam) untuk pekerjaan sejenis yang bersifat mengulang — misalnya bila di kelas membuat CRUD untuk 2 tabel, di luar kelas kerjakan CRUD untuk 3 tabel lainnya.

Tahap demi Tahap

Tahap 1 — Definisikan konteks dan aturan akses (30 menit)

- ☐ Lengkapi kelas `TenantContext` dengan metode `set()`, `clear()`, `has()`, `tenant()`, dan `id()`.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Susun daftar tabel dan model yang wajib tersaring per tenant, lalu catat hasilnya sebagai matriks akses. Proyek pendamping menyimpan keputusan ini pada `docs/adr/0008-tenant-isolation.md` dan matriks allow/deny-nya pada `evidence/pertemuan-04/security-verification.md`.
- ☐ Tentukan cara resolver menetapkan tenant aktif pada route internal: dari parameter route, dari host, atau dari pilihan tenant milik pengguna.
- ☐ Tentukan jalur mana saja yang boleh melewati global scope — misalnya katalog publik dan laporan admin platform — beserta filter pengganti yang wajib dipakai masing-masing.

Amati — Hasil yang kita harapkan

Setiap bypass mempunyai filter pengganti dan alasan. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Tabel keputusan ini menjadi acuan yang dapat diuji. Tanpa daftar eksplisit, pertanyaan "boleh atau tidak" dijawab berbeda-beda di tiap controller, dan celah baru muncul setiap kali fitur ditambahkan. Bagus, fondasi tahap ini sudah terbentuk.

Tahap 2 — Implementasikan TenantContext (30 menit)

Sampaikan — Apa dan mengapa

`TenantContext` menyimpan satu tenant aktif selama satu request atau satu job, dan gagal cepat bila belum diisi. Kita akan membangun kelas itu beserta pendaftarannya di container. Langkah ini penting karena seluruh lapisan berikutnya — scope, policy, dan binding — membaca tenant aktif dari satu sumber ini.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Buat kelas `TenantContext` pada `app/Support/Tenancy/TenantContext.php`, lalu daftarkan sebagai scoped binding di metode `register()` pada berkas `app/Providers/AppServiceProvider.php`.
- ☐ Mari kita buat `set`, `clear`, `has`, `tenant`, dan `id`.
- ☐ Pastikan setiap queue job membawa `tenant_id` pada payload-nya, bukan mengandalkan context yang sudah terbentuk, karena job dieksekusi di proses terpisah.

Amati — Hasil yang kita harapkan

Dua request/job tidak berbagi state tenant. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Keamanan tenant tidak bergantung pada satu pagar. Context menentukan tenant aktif, scope membatasi query, policy memeriksa izin, dan scoped binding mencegah model anak keluar dari induknya. Mantap, kita sudah menutup satu risiko penting.

Pro Tip untuk Artisan

Jalankan test isolasi tenant dengan filter paling kecil lebih dahulu. Feedback yang cepat membuat investigasi jauh lebih menyenangkan.

Tahap 3 — Middleware resolver dan role check (30 menit)

Sampaikan — Apa dan mengapa

Route tenant menjadi sumber tenant aktif, dan pengguna wajib memiliki role pada tenant tersebut. Kita akan menulis middleware yang memeriksa keduanya sebelum request diteruskan. Langkah ini penting karena pemeriksaan yang dilakukan setelah controller berjalan sudah terlambat.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Baca parameter `{tenant:slug}` dari route, lalu muat tenant yang bersangkutan di dalam berkas `app/Http/Middleware/SetTenantContext.php`.
- ☐ Periksa status tenant dan role pengguna pada tenant itu; tolak dengan 403 bila tenant nonaktif atau pengguna tidak memiliki role di sana.
- ☐ Isi context sebelum memanggil `$next($request)`, lalu bersihkan kembali pada blok `finally` agar nilainya tidak tertinggal.
- ☐ Daftarkan alias middleware tersebut pada berkas `bootstrap/app.php`.

Amati — Hasil yang kita harapkan

Tenant yang suspended atau tanpa role mendapat 403. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Urutan middleware menentukan segalanya di sini. Resolver harus berjalan setelah autentikasi — agar identitas pengguna sudah diketahui — tetapi sebelum route model binding, agar model yang dimuat sudah tersaring oleh tenant yang benar. Kerja bagus, hasilnya kini dapat kita uji dengan lebih percaya diri.

Tahap 4 — Global scope dan model integration (30 menit)**Sampaikan — Apa dan mengapa**

Trait `BelongsToTenant` menambahkan filter `where tenant_id` pada setiap query model sekaligus mengisi `tenant_id` secara otomatis saat data dibuat. Kita akan memasangnya pada seluruh model tenant-owned. Langkah ini penting karena filter yang dipasang di trait berlaku otomatis, sedangkan filter yang ditulis manual di tiap query cepat atau lambat akan terlewat.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Pasang trait `BelongsToTenant` pada model Menu, Category, Modifier, TenantOrder, OrderItem, Withdrawal, dan model tenant-owned lainnya di folder `app/Models/`.
- ☐ Larang pengisian `tenant_id` melalui mass assignment dari request — yaitu pengisian atribut model langsung dari input pengguna — agar nilainya hanya berasal dari context.
- ☐ Tambahkan pemeriksaan yang menggagalkan operasi tulis internal ketika context belum terisi, bukan membiarkannya menyimpan data tanpa `tenant_id`.

Amati — Hasil yang kita harapkan

Query biasa hanya mengembalikan tenant aktif. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Trait ini bekerja pada dua arah sekaligus: saat membaca, global scope mempersempit query ke tenant aktif; saat menulis, event creating mengisi ``tenant_id`` dari context. Keduanya membaca sumber yang sama, sehingga data yang ditulis pasti terbaca kembali oleh tenant yang sama. Sip, alur ini sekarang lebih mudah ditelusuri.

Tahap 5 — Policy dan scoped route binding (30 menit)**Sampaikan — Apa dan mengapa**

Policy memeriksa kemampuan dan role aktor terhadap sebuah objek, sedangkan scoped route binding menjaga agar resource anak hanya dapat ditemukan di bawah induk yang benar. Kita akan memasang keduanya. Langkah ini penting karena binding membatasi objek mana yang dapat dimuat, sementara policy membatasi tindakan apa yang boleh dilakukan terhadap objek itu.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Buat `MenuPolicy`, `TenantOrderPolicy`, dan `WithdrawalPolicy` di folder `app/Policies/`.

- ☐ Coba sunting item milik tenant lain melalui URL; hasilnya harus 403 atau 404, bukan halaman edit.
- ☐ Aktifkan `scopeBindings()` pada grup route bersarang dan pastikan nama relasinya tepat, agar binding anak benar-benar dibatasi induknya.
- ☐ Audit setiap Livewire action, karena parameter komponen juga merupakan input pengguna dan harus melewati policy yang sama.

Amati — Hasil yang kita harapkan

Mengubah ID pada URL atau payload Livewire tidak membuka data lain. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Perhatikan pembagian tugas keduanya. Scoped binding menjawab pertanyaan "objek ini milik siapa" dan menghasilkan 404 bila objek berada di luar induknya. Policy menjawab "aktor ini boleh melakukan apa" dan menghasilkan 403. Keduanya diperlukan karena objek yang sah dimuat pun belum tentu boleh diubah. Hebat, increment kita mulai terlihat seperti fitur yang utuh.

Tahap 6 — Public query dan bypass terkontrol (30 menit)

Sampaikan — Apa dan mengapa

Katalog pelanggan justru harus menampilkan menu dari banyak tenant sekaligus, sehingga query-nya perlu melewati global scope memakai `withoutGlobalScopes()` lalu menyaring canteen dan status secara eksplisit. Kita akan menempatkan query semacam ini pada satu kelas khusus. Langkah ini penting karena bypass yang tersebar di banyak controller praktis mustahil diaudit.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Buat kelas query khusus katalog publik, misalnya `app/Modules/Catalog/Services/PublicCatalogQuery.php`, dan pusatkan seluruh bypass scope di sana.
- ☐ Tambahkan aturan static analysis atau checklist review yang menandai setiap pemakaian `withoutGlobalScopes()` di luar kelas tersebut.
- ☐ Catat ke log setiap akses lintas tenant yang dilakukan admin platform, lengkap dengan aktor dan lingkup datanya.

Amati — Hasil yang kita harapkan

Katalog hanya menampilkan tenant aktif dari canteen hasil QR. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Ingat bahwa `withoutGlobalScopes()` tidak memberi izin apa pun — ia hanya mengubah bentuk query. Karena itu urutannya wajib: peroleh kantin atau tenant yang sudah terotorisasi lebih dahulu, baru lepaskan scope, lalu segera pasang filter pengganti. Hampir selesai—quality gate akan membantu kita memastikan semuanya tetap sehat.

Tahap 7 — Security tests dan commit (30 menit)

Sampaikan — Apa dan mengapa

Kasus yang ditolak sama pentingnya dengan kasus yang berhasil. Kita akan menulis security regression test — test yang sengaja mencoba akses terlarang dan memastikan aplikasi menolaknya. Langkah ini penting karena celah isolasi biasanya muncul kembali diam-diam saat fitur baru ditambahkan.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Tulis test yang memastikan Tenant A tidak dapat membaca maupun mengubah data Tenant B melalui index, detail, update, delete, dan route binding; simpan di `tests/Feature/TenantIsolationTest.php`.
- ☐ Tambahkan test yang memastikan query katalog publik tidak membocorkan data antarkantin.
- ☐ Tambahkan test yang memastikan job membentuk context sendiri dan membersihkannya kembali setelah selesai, sehingga dua job berurutan tidak berbagi tenant.
- ☐ Eksekusi commit di terminal dengan pesan `feat(tenancy): enforce tenant isolation layers`.

Amati — Hasil yang kita harapkan

Seluruh matriks allow/deny hijau. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Test adalah requirement yang dapat dieksekusi. Feature test memeriksa perilaku, integration test memeriksa kontrak antarkomponen, dan E2E membuktikan alur pengguna dari awal sampai akhir. Selamat! Increment minggu ini sudah siap dibuktikan dan di-commit.

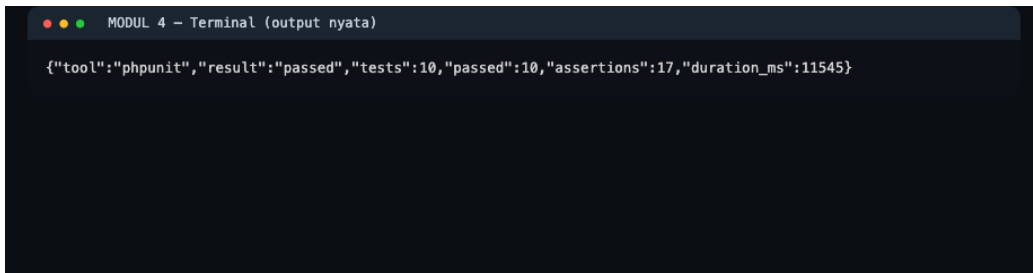
Guided Lab: Predict → Try → Observe → Explain

TRY · Eksekusi perintah berikut di terminal, berurutan dari atas ke bawah, satu baris pada satu waktu. Berhenti saat muncul error yang belum dipahami; gunakan tabel troubleshooting sebelum melanjutkan.

```
Lokasi: terminal pada root repository kantin-multi-tenant • MODUL 4 – TERMINAL • BASH
php artisan route:list --path=tenant
php artisan migrate:fresh --seed
php artisan test --filter=TenantIsolation
php artisan test
./vendor/bin/pint --test
```

Output Nyata Terverifikasi

Matriks isolasi tenant (scope, scoped-binding, policy, bypass, job) hijau.



```
{"tool":"phpunit","result":"passed","tests":10,"passed":10,"assertions":17,"duration_ms":11545}
```

Gambar M4. 10 uji isolasi tenant lulus (60 uji total, 132 assertion; phpstan 0 error, pint lulus). Matriks allow/deny 12 skenario tercatat pada *evidence/pertemuan-04/security-verification.md* di repository.

Observe: Cocokkan dengan Checkpoint

- ☐ Setiap bypass mempunyai filter pengganti dan alasan.
- ☐ Dua request/job tidak berbagi state tenant.
- ☐ Tenant yang suspended atau tanpa role mendapat 403.
- ☐ Query biasa hanya mengembalikan tenant aktif.
- ☐ Mengubah ID pada URL atau payload Livewire tidak membuka data lain.

Explain: Ucapkan dengan Istilah Sendiri

- Mengapa global scope saja belum cukup?
- Kapan bypass scope sah?

Cuplikan Kode dan Penjelasan

Rujukan pendamping: berkas `04_tenancy/TenantContext.php`, `BelongsToTenant.php`, `SetTenantContext.php`, dan `TenantIsolationTest.php`. Di repository nyata: `app/Support/Tenancy/TenantContext.php` (didaftarkan scoped pada `app/Providers/AppServiceProvider.php`), trait pada `app/Models/Concerns/BelongsToTenant.php`, middleware pada `app/Http/Middleware/SetTenantContext.php`, dan test pada `tests/Feature/TenantIsolationTest.php`.

```
static::addGlobalScope('tenant', function (Builder $builder): void {
    $context = app(TenantContext::class);

    if ($context->has()) {
        $builder->where(
            $builder->qualifyColumn('tenant_id'),
            $context->id()
        );
    }
});
```

```
static::creating(function ($model): void {
    if (app(TenantContext::class)->has()) {
        $model->tenant_id = app(TenantContext::class)->id();
    }
});
```

- ☐ `qualifyColumn()` menghindari kolom ambigu pada join.
- ☐ Scope membaca binding scoped container, bukan variabel global proses.
- ☐ Auto-fill membantu konsistensi, tetapi policy dan database constraint tetap wajib.

Cuplikan menonjolkan pola inti. Guided Lab memberi urutan pembuatan, verifikasi, dan bukti; lengkapi relasi, factory, namespace, serta binding sesuai increment yang telah dibangun.

Verifikasi dan Kriteria Selesai

- ☐ Context dibersihkan setelah request.
- ☐ Model scoped menyaring tenant.
- ☐ Policy menolak URL/payload silang.
- ☐ Public catalog difilter canteen.
- ☐ Test matrix hijau.

Jika Macet: Troubleshooting Cepat

Tiga gejala berikut paling sering muncul saat lapisan isolasi mulai dipasang. Cocokkan pesan error atau perilaku yang Anda temui dengan kolom Gejala, lalu ikuti perbaikannya sebelum melanjutkan ke tahap berikutnya.

Gejala	Penyebab mungkin	Perbaikan	Bukti pulih
TenantContext kosong	Urutan middleware salah	Resolve setelah auth sebelum binding	Request membaca tenant aktif
Data tenant lain terlihat	Scope atau policy terlewat	Terapkan pertahanan berlapis	Cross-tenant test ditolak
Queue memakai tenant lama	Context tidak dibersihkan	Clear context pada finally	Dua job berurutan terisolasi

Tantangan Bertingkat

LEVEL INTI · Pastikan objek tenant lain menghasilkan 403 atau 404.

LEVEL LANJUT · Uji dua job tenant pada worker yang sama.

Evidence Praktikum

- ☐ TenantContext request-scoped dan middleware resolver.
- ☐ Trait `BelongsToTenant` pada model tenant-owned.
- ☐ Policy dan scoped route binding untuk menu/order/withdrawal.
- ☐ Test matrix allow/deny tenant A vs tenant B.

Exit Ticket — Jawab Sebelum Commit

1. Mengapa global scope saja belum cukup?

Jawaban:

2. Kapan bypass scope sah?

Jawaban:

3. Bagaimana mencegah context bocor pada queue worker?

Jawaban: